

Payoff

You get rid of unnecessary duplicated code, making the resulting code
.less bulky

Code becomes more readable and understandable (you no longer have to guess the reason for creation of a second class performing the exact same functions as the first one)

When to Ignore

Sometimes merging classes is impossible or so difficult as to be pointless. One example is when the alternative classes are in different libraries that each have their own version of the class

Alternative Classes with Different Interfaces

علائم و نشانه‌ها (Signs and Symptoms)

دو کلاس که عملکرد یکسان دارند ولی اسم متدهایشون فرق می‌کنه.

دلایل ایجاد (Reasons for the Problem)

برنامه‌نویسی که یکی از کلاس‌ها رو ساخته، احتمالاً نمی‌دونسته که یه کلاس مشابه از قبل وجود داشته.

مثال

csharp

```
// دو کلاس با عملکرد یکسان، اسم‌های متفاوت
public class FileReader
{
    { /* ... */ } public string Read(string path)
    {
```

```
public class DocumentLoader
{
    // { /* ... */ } public string Load(string filePath)
    {
        Read رو میکنه!
```

درمان (Treatment)

#	تکنیک	کاربرد
۱	Rename Methods	اسم متدها رو توی هر دو کلاس یکسان کن
۲	Move Method / Add Parameter / Parameterize Method	امضا و پیاده‌سازی متدها رو یکسان کن
۳	Extract Superclass	اگه فقط بخشی از عملکرد مشترکه، والد مشترک بساز (قبلاً کار کردیم )
۴	حذف یکی از کلاس‌ها	بعد از یکسان‌سازی، یکی رو حذف کن

Payoff (دستاورد)

- حذف کد تکراری غیرضروری، کد کم‌حجم‌تر
- کد خواتر و قابل فهم‌تر (دیگه لازم نیست حدس بزنی چرا یه کلاس دوم با همون عملکرد ساخته شده)

When to Ignore

گاهی ادغام کلاس‌ها غیرممکن یا بی‌فایده‌ست. مثلاً وقتی کلاس‌های جایگزین توی کتابخونه‌های مختلف هستن که هر کدوم نسخه خودشون رو دارن.

تمرین 

این کد رو با درمان مناسب ریفتور کن:

csharp

```

        public class UserRepository
        {
            public User GetById(int id) { /* ... */ return new User(); }
            { /* ... */ } public void Insert(User user)
            { /* ... */ } public void Remove(int id)
        }

        public class AccountService
        {
            !GetById همون // public User Fetch(int userId) { /* ... */ return new User(); }
            !Insert همون // { /* ... */ } public void Create(User account)
            !Remove همون // { /* ... */ } public void Delete(int accountId)
        }

```

راهنمایی:

- متدها رو **Rename** کن تا یکسان بشن
- یکی از کلاس‌ها رو نگه دار، اون یکی رو حذف کن

بفرست تا بررسی کنیم! 📩